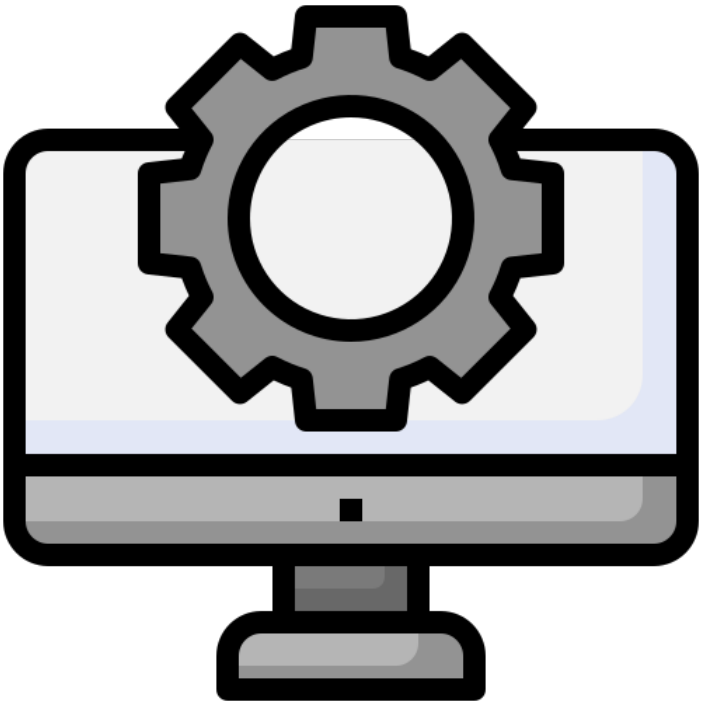
Génie Logiciel

GeoHealth

Architectural Justification



Date: 2025-2026

Authors: Anas Damlakhi, Julien Jamoulle, Ilias Essarhiri, Houria Ariouat, Thomas Busoni, Louis Bomal

1 Preface

1.1 What is this document?

This document is an **explanation of the architectural strategy** chosen for the GeoHealth software, developed as part of the *INFOM114* course at the University of Namur.

In this document, you will find a brief description of the software architecture, an explanation of why certain architectural choices were made over others, and a presentation of the technical stack used, along with the reasons behind those choices.

This document does not cover all implementation details of the software, as there is a lot to discuss. If you would like more detailed information about a specific package, we strongly recommend consulting the project's technical documentation, which explores the project in greater depth.

1.2 How is this document organized?

This document is divided into three main parts:

- **Preface** (which you are currently reading), which explains the purpose and organization of this document.
- **General Architecture section**, which explains the architectural pattern chosen for this project, an alternative that was considered but not selected, and the main strengths and weaknesses of the chosen architecture.
- **Layer Explanation section**: as the architecture is based on layers, this section describes each layer, the technologies used, the reasons behind these choices, and provides an overview of their internal organization.

1.3 Acknowledgements

The spelling of this document was reviewed with the help of Reverso, ChatGPT, and Claude, under our supervision, as English is not our native language.

Contents

2 General Architecture

This section will present the general architecture of the GeoHealth software and explains why we believe it is the best option. It will also highlight some inherent weaknesses of the architecture in oder to provide a more nuanced explanation.

2.1 What is GeoHealth software ?

Before we go any further, I'd like to give you a brief explanation about GeoHealth software. As I think it's helpful to have at least a basic understanding of what GeoHealth is to fully grasp its architecture.

Geohealth is a web GIS. This means that it is a website that aims to display, analyse, and share maps and spatial data.

In the case of Geohealth, its main objective is to display risk maps for two diseases: Rift Valley fever and Ebola, and to collect geographical experts' opinions about those maps through an accessible interface.

Therefore, one of the key architectural constraints was that the application had to be web-based, which reduced the scope of possible architectures.

2.2 Chosen Architectural Pattern

The architectural pattern we chose for Geohealth is a **multi-layer architecture**.

The main principle of this architecture is to divide the application into several components that are grouped together into what we call layers having all their own purpose.

Of course, the layers should be as loosely coupled as possible to the others to fully benefit from this architecture.

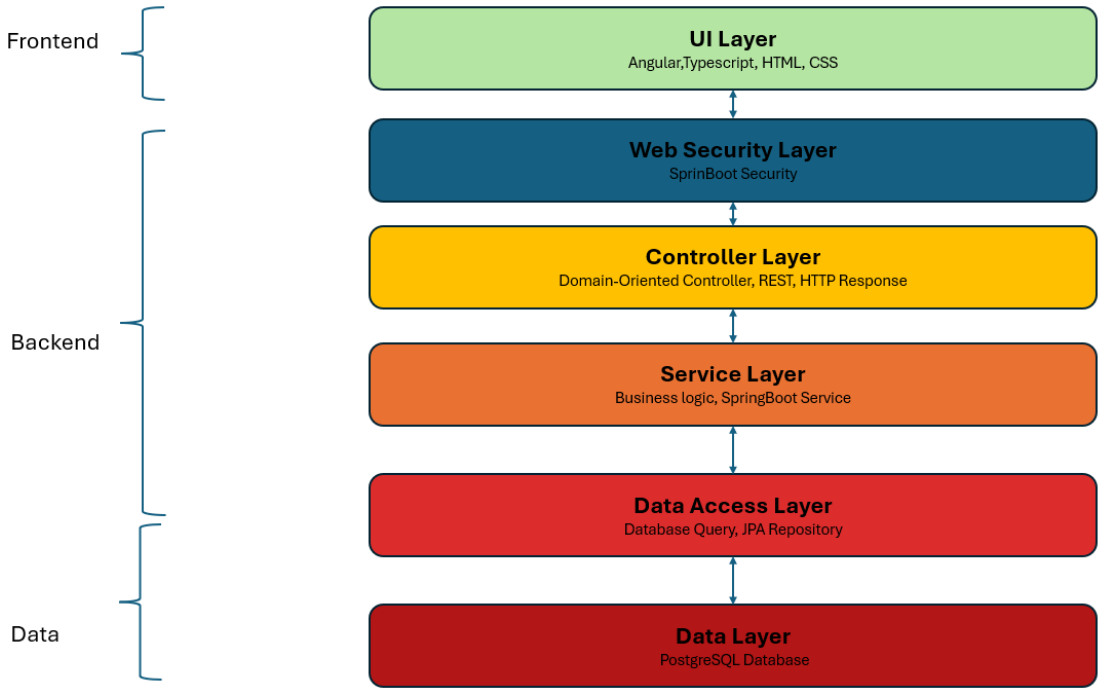


Figure 1: Architecture of GeoHealth

Here is a small schema that gives you a view of the different layers of the GeoHealth architecture. As you can see, each layer is only dependent on one other layer.

The purpose and the technology used to implement these layers will be discussed in the Layer Explanations section.

2.2.1 Why we chose this architecture

The question you may ask is why we chose a layered architecture. Here are some of the reasons why we chose it

- The layered architecture is the most used architecture on the web. Therefore, there is a lot of robust frameworks that already implement this architecture, making it easy to adopt.
- As each layer has its own logic and is not directly dependent of the implementation of the other layer, this allows to split the task more easily among the development teams and avoid conflicts.
- The members of the development teams were already familiar with this architectural style, making it easier to keep the big picture in mind while working and simplifying the learning process.

- The layered architecture is well known for simplifying maintainability because as long as a layer keeps its service consistent, it can change its internal structure without impacting other layers

2.2.2 Weaknesses

Of course, we know no architecture is perfect. This architecture has two well-known drawbacks

- Complexity: Indeed, as each layer has its own logic and its own organisation, it can introduce a lot of complexity.
- Overhead: Each layer add it's own processing on top of the others, which can result in huge overhead if the communication between the layers is not well managed.

2.3 Another architecture considered

The micro services architecture was initially considered as an option, but it was rejected due to several factors

- Most of the members of the teams were not familiar with this architecture, which would have added a useless overhead during development.
- GeoHealth has mainly one functionality, which is managing maps, which is too small to fully benefit from a micro services approach.

3 Layer Explanations

This section will go into each layer to give a general overview of the layer organization and explain which technology was used to implement this layer, and why.

3.1 Data

3.1.1 Technologies used and why they were chosen

For the data part, the following technologies have been used:

- **PostgreSQL**: PostgreSQL is a widely used relational database in Spring Boot applications. Furthermore, it offers an extension called PostGIS dedicated to geographical data management, which is very interesting for our project.
- **JPA ORM (hibernate implementation)**: JPA is directly integrated into SpringBoot (Spring Data JPA) and well-documented online. Futhermore we wanted to use ORM paradigm as it allows us to abstract the complexity of PostgreSQL and to remain within the Java object-oriented paradigm at all times. Therefore, reducing the potential number of errors.

3.1.2 Data Layer Overview

Role: Storing all the data of the project

The data layer is largely abstracted thanks to the use of an ORM This means that most database interactions are expressed directly in Java code, which is automatically translated into SQL queries.

Therefore, in reality, the database is generated based on a set of classes that are marked with an `@Entity` tag telling to Springboot that this class should represent a database table. The mapping behind it is pretty simple. A class is equivalent to a table and each of the class attribute become collumns of the SQL Database.

But to at least give you a look at what the databse look like and not just a set of classes. The following schema represents the database and links each table to its associated Springboot entity

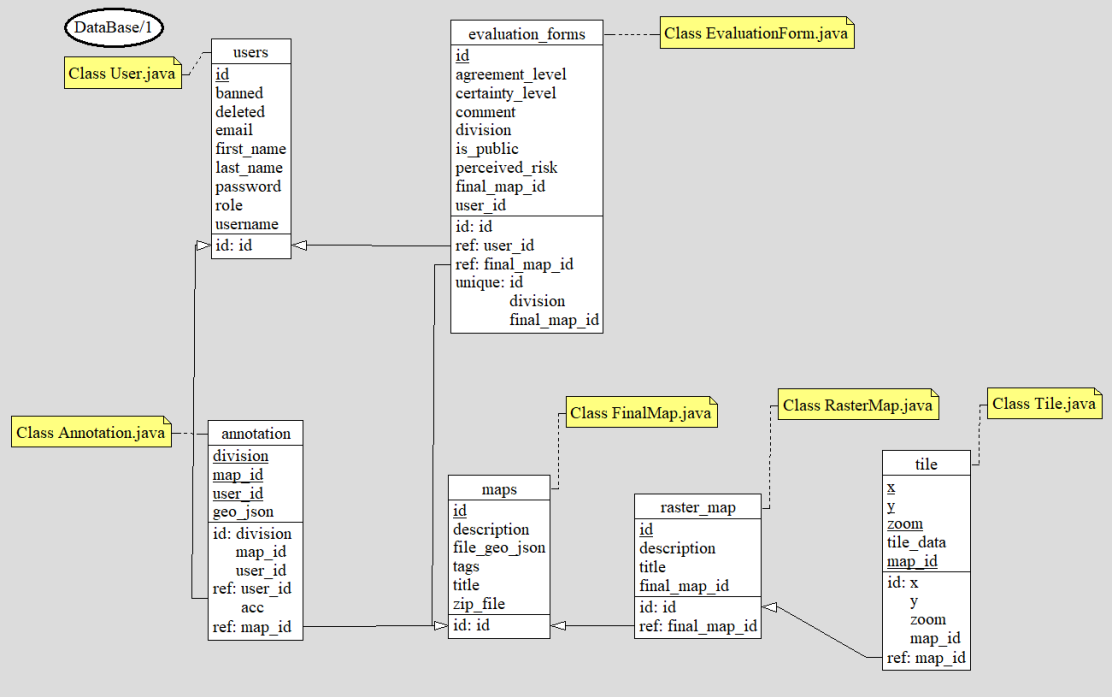


Figure 2: Database of GeoHealth

3.1.3 Data Access layer Overview

Role: Offering an easy and standardized access to the data of the project

This layer is also very simple to explain, as it's also largely abstracted by spring-boot. Indeed for each Spring Entity you can define a Spring Data Jpa interface, which provide a set of common database operations directly accessible through java function avoiding us to perfom database operation manually

```
package com.webgis.user;

import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Integer> {
    Optional<User> findByUsernameAndDeletedFalse(String username);
    Optional<User> findByEmailAndDeletedFalse(String email);
    Optional<User> findByIdAndDeletedFalse(Long id);
    List<User> findAllByDeletedFalse();
}
```

Here is an example of the repository that is used for managing the user in our project. All the functions you see here are existing ones and can be directly used

without having to code them.

It represents a significant gain in development time, as developers do not need to repeatedly write SQL queries manually. Additionally, it improves the robustness and maintainability of the codebase, since Spring Data JPA is a mature and widely adopted library within the Spring Boot ecosystem.

3.2 Backend

3.2.1 Technologies used and why they were chosen

For the backend part, the following technologies has been used:

- **Spring Boot**: Spring Boot was chosen primarily because of its large and rich ecosystem and its extensive documentation, In addition to the many tutorials available online. This made it significantly easier to find solutions to problems encountered during development, such as database integration, security and REST API design. Additionally, Spring Boot is widely adopted in the industry, which was an important factor for the team, as we treated GeoHealth as a project that could realistically be integrated into a real professional environment.
- **BCrypt**: Used for password hashing, BCrypt was chosen as it is the industry standard for securely storing passwords.
- **JWT**: Used for stateless authentication, allowing the server to verify the identity of a user without storing any session information, which fits well with the REST API approach.

3.2.2 Service layer Overview

Role: Contains all the business logic of the application.

The Service layer sits between the Controller layer and the Data Access layer. It contains all the rules that define what the application does, independently of how the data is stored or how requests arrive.

This separation is very useful as it isolates the business logic into dedicated service classes. Each domain of the application has a single, clearly defined place where its rules are enforced, making the code more readable and maintainable through a clear separation of concerns.

3.2.3 Controller Layer Overview

Role: Handles all incoming web requests and returns the appropriate responses.

The Controller layer is the entry point of the backend for user interactions. It receives HTTP requests from the frontend, uses DTOs to structure the incoming

data, and forwards it to the Service layer. It is not responsible for any business logic. Its purpose is to extract the needed data from the request, forward it to the Service layer, and translate the result into an appropriate HTTP response.

3.2.4 Web Security Layer Overview

Role: Controls access to the application's endpoints, ensuring that only authorized users can perform the operations they are permitted to.

The Web Security layer intercepts all requests before they reach the Controller layer. Every incoming request passes through it before reaching any controller, allowing authentication and authorization logic to be enforced in a single, centralized place, completely separated from the business logic.

This interception serves two purposes: access control, where certain endpoints are restricted to specific roles or require authentication, and stateless authentication, where instead of relying on server-side sessions, a JWT token is extracted from each request and validated to identify the user making the request.

3.3 Frontend

3.3.1 Technologies used and why they were chosen

For the frontend part, the following technologies have been used:

- **Angular**: Angular was selected as the frontend framework due to its component-based architecture, which is well-suited for building structured and maintainable single-page applications. We also chose Angular as one of the development team members had already used this framework.
- **TypeScript**, **Html**, **CSS**: When choosing the Angular framework, it directly implies using TypeScript. For us, it was not a problem as we believe that typed languages reduce errors and facilitate overall code comprehension. Of course, Angular also uses HTML and CSS, but these technologies are fundamental and widely adopted standards in web development. Therefore, this does not present any issue.

3.3.2 UI Layer Overview

Role: Displaying a graphical interface for the user and managing its interaction.

The UI Layer is more complex than other layers due to the choice of the Angular framework. Indeed, Angular already provides a skeleton that contains many pre-existing files, each serving a specific purpose.

However, the frontend implementation can be divided into 3 main parts

- **core**: This part is responsible for managing all calls made to backend services and API endpoints
- **feature**: This part contains all the main pages of the website, including their HTML, CSS, and TypeScript code.
- **shared**: This part contains all the components (such as buttons, navigation bars, and form inputs) and data structures (Data Transfer Objects) that are shared among all the pages of the website

Of course, all of these folders interact with one another to provide the best possible experience. These interactions can be summarized by the following diagram.

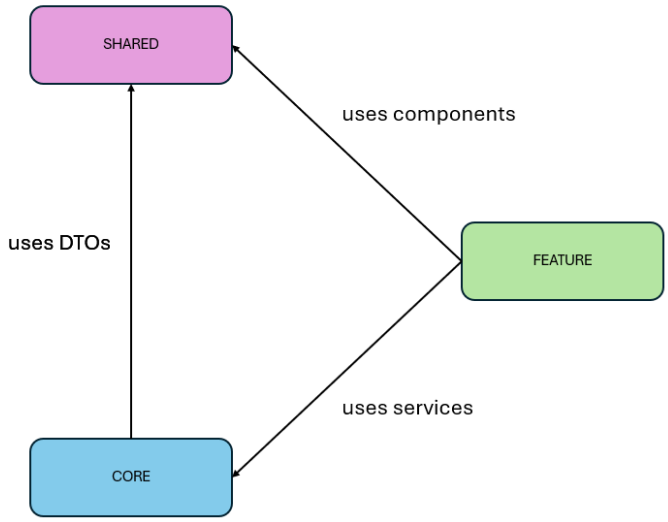


Figure 3: Relationship among the UI layer

For us, this division into three parts is great as it clearly separates concerns across different folders, it also promotes sharing among the different webpages, and clearly defines the relationships that exist within this layer.

3.4 CI/CD

3.4.1 Technologies used and why they were chosen

For the CI/CD pipeline, the following technologies have been used

- **GitHub Actions**: GitHub Actions was the our choice as our project is hosted on Github. Github actions was the natural choice, it required no external tools.
- **SonarCloud**: SonarCloud was chosen as the code quality analysis tool due to its easy integration with GitHub Actions and the team's familiarity with it, allowing us to automatically detect vulnerabilities and code smells on every pull request.

- **Nginx**: Nginx was chosen as the web server on the production VM due to it being an industry standard. It serves as the single entry point for all incoming traffic, serving the Angular static files and acting as a reverse proxy for the backend.

3.4.2 Pipeline Overview

GeoHealth had to be continuously deployed and accessible as it is web GIS. To achieve this goal, the pipeline has been divided into two concerns: ensuring nothing breaks the build before a merge, and automatically deploying when it does.

3.4.2.1 Ensuring nothing breaks before a merge

To meet this objective, a Checkstyle analysis, SonarCloud analysis, and automated tests are run on every pull request targeting the main branch to verify that both existing and new functionalities do not break the build.

3.4.2.2 Automatically deploying

On merge requests into the main, the deployment triggers. At that point, GitHub Actions builds both the Angular frontend and the Spring Boot backend, and transfers the compiled artifacts directly to the university server (the Angular static files and the Spring Boot JAR) which are then simply run on the server without any further compilation needed.

This was done to keep the server as lightweight as possible, as all the heavy building work is handled by GitHub Actions rather than the server itself, meaning the server only needs to run the already compiled artifacts.